

Retiming over CppHDL Synthesis

Retiming operates on the shared operation graph in `synth/` without modifying the C++ source. Both the original and retimed graph can be simulated as native C++, and the synthesis driver maps the result to gate-level Verilog.

For `hls::Clocked`, use `--synth`. The synthesis driver converts the actual AST schedule directly into typed operations, registers and memory ports in the shared graph. It never lowers the native `Clocked::_work` reference, which has different latency from the scheduled FSM. The wrapper type selects delayed or pipeline scheduling automatically, with or without synthesis. Ordinary conversion emits only RTL and does not run graph export. Neither flow requires Yosys: synthesis uses the exporter and generic gate mapper in `synth/`.

There are two different contracts:

Mode	Existing clock-cycle behaviour	Register count	Purpose
<code>keep_behaviour_retiming</code>	Reserved	May change when banks split or merge	Balance existing stages
<code>fit_pipeline_retiming</code>	Additional latency; transaction feedback may reduce throughput	Increases	Meet an estimated stage period

Retiming is opt-in. Without a command-line rule or a source annotation, synthesis reports estimated timing but does not move or insert registers.

Delay Estimates

`synth/timing.h` defines `DelayModel`; `synth/timing.cpp` evaluates it. All values are nanoseconds. Defaults are intentionally simple:

Component	Estimate
AND / OR	0.05
XOR	0.10
Multiplexer	0.10
Register clock-to-Q / setup	0.10 / 0.05
Add, subtract, ordered compare	0.05 + 0.06 per operand bit
Equality / reduction	Gate-tree estimate proportional to $\log_2(\text{width})$
Variable shift	Multiplexer-tree estimate proportional to $\log_2(\text{width})$
Multiply	$\log_2(\text{width})$ estimated adder levels
Divide / remainder	width estimated adder levels
Memory read	$0.50 + 0.08 * \text{ceil}(\log_2(\text{depth}))$

Constant shifts and bit rearrangement are wiring. Memory write address, data, and enable paths terminate at a setup-time boundary. Registered memory reads include both the memory delay and the destination register's setup requirement. The analyser checks input-to-register, register-to-register, register-to-output, and memory-write paths. It accounts for mux/select paths, not only data inputs.

Use `--delay-scale 1.5` to scale every estimate, including register overhead. C++ users of the synthesis API can supply a custom `DelayModel`.

These are **estimates, not static timing analysis**. They do not model a target library, placement, routing, fanout, clock skew, hold time, or reset recovery and removal. C++ integer promotions can make estimates conservative. Frequencies declared by clock options do not select a retiming target automatically: specify the period explicitly. Final acceptance still requires technology mapping and timing analysis with the actual constraints.

Apply a Rule

To pipeline the whole design toward a 2.5 ns estimated period:

```
1 build/cpphdl --synth --top cpphdl_top --module SynthRetiming \  
2   --retiming fit_pipeline_retiming --clock-period-ns 2.5 \  
3   --output /tmp/retimed synth/tests/retiming/logic_retiming.cpp
```

To limit the rule to one C++ module instance and its descendants:

```
1 build/cpphdl --synth --top cpphdl_top --module SynthRetiming \  
2   --retiming keep_behaviour_retiming --clock-period-ns 2.5 \  
3   --retime-module cpphdl_top.stage \  
4   --output /tmp/retimed-stage synth/tests/retiming/logic_retiming.cpp
```

The selector is an **instance path**, not a class name. A misspelled or empty region is an error. Source annotations apply a rule to each instance of a class:

```
1 class [[clang::annotate("CPPHDL_RETIMING=fit_pipeline_retiming:2.5")]]  
2 ArithmeticChain : public cpphdl::Module {  
3     // Ordinary ports, registers, _work and _strobe methods.  
4 };
```

The format is `CPPHDL_RETIMING=mode:period_ns`. Protect the attribute with `#ifdef __clang__` when another native compiler warns about unknown attributes. An explicit CLI retiming rule overrides source rules for that invocation. Multiple disjoint annotated instances are allowed; overlapping rules are rejected instead of retiming the same region twice. State names and operation scope metadata survive lowering; common-expression merging does not cross scope boundaries.

Keep Behaviour

This mode searches for moves across pure combinational operations:

- Forward: compute an operation before the register boundary, replacing banks at its operands with a bank at its result where those operand banks are unused.
- Backward: move a result bank to the operation's operands and compute the operation after those banks.

Each move preserves the number of clock boundaries along the affected paths. Only moves that improve the selected region's worst estimated path are accepted. Unrelated consumers keep the storage they still need. Native initial values are transformed too; backward moves requiring an unsupported initial-value inverse are not attempted. Synthesized power-up state remains unspecified unless the design resets it.

Clock, edge, reset, and memory boundaries constrain legal moves. Forward moves require compatible clock/edge/reset domains and transform constant asynchronous reset values. Backward movement across asynchronous-reset registers is currently disabled. Registers do not move through a RAM access or across clock domains.

This is a bounded greedy search, not a globally optimal retiming solver. A target may be impossible with the existing latency, or this search may not find it. The driver still emits the improved design and sets `target_met` to false. It must not insert extra cycles to disguise that failure.

Fit a Pipeline

This mode traverses the feed-forward dependency graph and inserts register banks when another operation would exceed the available stage delay. It adds matching delays to the other operands of a join, including mux controls. Existing feed-forward register stages retain their original relative cycle boundaries. Externally visible selected register outputs are balanced to the same added latency, so a tag carried alongside data stays aligned. Balancing output copies does not add delay to an internal register's other consumers.

Feed-forward graphs accept one transaction every clock. Carry validity, frame IDs and errors as data alongside the payload so they receive matching delays. Feedback graphs use the ready/commit contract below instead.

Inserted stages preserve a common constant synchronous reset, or the selected domain's constant asynchronous reset. Reset control is not delayed with data. Pipeline contents are invalid during filling after reset; discard the reported added latency before comparing transactions. The source C++ model keeps its original latency. Use the generated native graph to simulate the transformed pipeline, or compare its outputs against delayed source-model expectations.

Current rejection conditions include:

- Multiple clock/edge/reset domains within one fit rule, or state dependencies crossing clocks or edges. Use separate rules for independent domains.
- Mixed synchronous-reset contracts within the selected region.
- Combinational top-level output boundaries; expose registered outputs first.
- A register whose added latency would change a memory write transaction.
- A selected register driving an unselected register. Include the downstream pipeline in the rule instead of changing only one side of that boundary.
- A memory address path that would need to be delayed.
- A single indivisible operation that exceeds the target by itself.

Fit mode expands an oversized arithmetic operator, including multiplication or division, into CppHDL’s generic gates before placing register boundaries. This permits registers inside the calculation, rather than only around it. Memory operations and explicit `keep_box / one_clock` regions remain intact. If an indivisible region or the transformed design still exceeds its target, fit mode fails and no successful gate netlist is reported. Gate-level delay estimates are not technology timing closure, and expansion can be expensive in both area and latency.

For a scoped fit rule, callers must account for the changed module latency. This is not a protocol-preserving transformation across arbitrary module boundaries. Expose registered module outputs at the synthesis boundary, or include their downstream consumers in the selected region.

Streaming HLS Feedback

`ClockedPipeline<T, STAGES>` selects a different contract from the feedback transactions below. HLS builds the initial pipeline from method dependencies, without delay estimates. Synthesis consumes its saved transition graph and stage boundaries; `fit_pipeline_retiming` adds timing-driven cuts and aligns operands, result/fault metadata, valid bits, and candidate next state. It retains the existing ready/valid ports and `II=1`. It does not add a `retiming_ready_out/retiming_commit_out` transaction adapter.

Each admitted call reads committed state. Its updated object snapshot commits when its result enters the last stage. Calls admitted before that commit, or on the commit edge itself, read older state. Extra timing stages therefore delay feedback and can change FSM behavior. This is intentional: use latency-tolerant feedback in `ClockedPipeline`, or select `ClockedDelay` when call-to-call ordering must be preserved. There is no automatic stale-state interlock or field-wise merge between overlapping calls.

Backpressure freezes the entire region. Bubbles cannot commit an update, and clocked reset cancels pending results and restores constructor state. The retiming report gives added latency, register bits, and `II`; streaming-region metadata and `timing.json`’s `streaming_regions` give the resulting total latency. A test must model that latency, not assume the original native wrapper’s `STAGES` still applies.

Rules may select a complete streaming region or its parent, not part of its internal pipeline. Any surrounding unmodified logic must also meet the target. The first stage’s timing bud-

get includes the arrival delay of parent logic driving the region's inputs. Retiming expands arithmetic operators to the generic mapper's gate network before placing internal boundaries, so those boundaries are checked against the implementation that will be emitted. `keep_behaviour_retiming` leaves a stream unchanged if timing already fits; otherwise it rejects rather than altering its feedback latency. Scheduled memory, asynchronous reset, and per-function keep-box/one-clock constraints inside this HLS pipeline mode are not yet supported. Cell delays remain estimates, not proof of a technology-specific frequency.

`synth_stream_checks` tests independent regions, scoped and repeated retiming, clock ownership, graph serialization and mapping. The `HLS Pipeline` and `PipelineFeedback` tests additionally run retimed mapped gates with Verilator.

Feedback Transactions

An ordinary RTL or delayed-HLS feedback graph evaluates one original clock transition as a transaction. This is not the `ClockedPipeline` policy. The retimer captures its inputs and keeps the original registers unchanged until all next values are available. It then commits **all** original registers on the same edge. Intermediate pipeline registers may update on other edges; they are not visible as committed state.

The generated design adds two outputs:

- `retiming_ready_out`: present a new input transaction on this edge. Inputs may change while this signal is low; their captured values are used instead.
- `retiming_commit_out`: this edge commits the current transaction. Read its result after the edge. These signals are evaluated before the edge.

`initiation_interval` is `added_latency + 1` for this schedule. For example, three inserted stages give an acceptance at edge 0, a commit at edge 3, and the next acceptance at edge 4. State feedback always reads the previous committed transaction. The scheduler does not consume old counter values to overlap dependent transactions. When no extra stages are needed, both signals can be high on every edge.

Reset cancels the in-flight transaction and suppresses ready/commit. Supply the design's reset before using synthesized hardware. Original output fields, including validity and error bits, remain held between commits: qualify them with the commit event, not with their level alone. There is no output-ready input; the consumer must accept every commit. This contract changes throughput as well as latency and must be connected explicitly by the caller.

Current feedback support requires that the selected region contain all graph registers in one clock/edge/reset domain. The ordinary C++ graph path still requires registered outputs and no memory. The scheduled HLS path additionally supports memory and combinational output bundles as described below. Independent feedback islands and an elastic output queue are not implemented.

Scheduled HLS Graphs

For this flow `--top` can name a top-level C++ module class or object variable:

```
1 build/cpphdl --synth --top ScheduledTop --module ScheduledTop \  
2   --retiming fit_pipeline_retiming --clock-period-ns 3.205128205 \  
3   --output /tmp/scheduled-retimed synth/tests/hls/ClockedSchedule.cpp
```

The output directory contains `graph.cc` (the shared-graph builder), `retimed_graph.cc`, the timing report and mapped `gates.v`. Export does not generate or reparse SV. Mapped combinational gates have individual bit outputs; module ports retain their original widths. External tool invocations have a 300-second default timeout. For a large design, `--tool-timeout SECONDS` changes that execution budget without changing the timing target or accepting an over-budget circuit.

In this flow, one retiming transaction represents **one original FSM edge**, not one complete method call. Present all inputs at `retiming_ready_out`. Sample the complete output bundle **before the edge where `retiming_commit_out` is high**. Only then interpret the original valid/ready signals. Output values outside that commit event are not protocol events and need not remain stable. This is an explicit transaction interface, not a drop-in retimed AXI or stream interface. A surrounding hardware adapter must obey that contract.

All memory writes and original state registers commit together. A delayed read therefore sees the same old memory contents as the original FSM edge. Captured inputs, registers and memory remain stable until commit, so values computed early do not need shift registers merely to wait for later branches. Registers are still inserted wherever required to break a long timing path. When branches arrive at different stages, the retimer can register an early branch during its spare cycles and reuse that value at later joins. It does not need a new copy of that register at every use of the branch.

The scheduled exporter currently accepts the default positive-edge `clk`, synchronous reset (including intentionally unreset storage), asynchronous memory read ports with explicit read registers, and whole-word writes per independent bank. Other clock domains and unsupported schedule expressions are rejected, rather than falling back to the native transaction reference. Cycle-local scratch starts at zero, as in the ordinary HLS emitter; application initialization still runs through its scheduled reset entry. Function-level `CPPHDL_ONE_CLOCK` annotations inside scheduled HLS methods are not yet supported.

The target is an estimated stage period, **not** an initiation interval of one clock. A dependent FSM may need many physical clocks per original edge. Check both `initiation_interval` and register count before accepting the result.

Keep a Function Within One Clock

Mark a pure function when its internal operations must not be split:

```

1 [[clang::annotate("CPPHDL_ONE_CLOCK")]]
2 uint8_t transform(uint8_t value) {
3     return uint8_t(value + 13) ^ 0x59;
4 }

```

Each call has its own timing boundary. The estimate includes the function’s live operations and helper calls. The retimer may register its inputs or outputs, but cannot insert a register inside the call. This annotation does not add a clock itself. It is checked against the selected retiming period, including register overhead; an oversized function fails with **one-clock function exceeds target period** in either retiming mode.

Functions must not modify registers, caller-owned variables or memory. Nested annotated calls and overlap with a kept module are rejected. Ordinary pure helper calls inside the function are allowed. Constant folding still applies. Unlike CPPHDL_KEEP_BOX, this annotation uses the cell-delay model rather than a user-supplied delay. It is a graph-synthesis constraint, not an HLS scheduler directive. Guard it with `#ifdef __clang__` for native compilers that warn about unknown attributes.

Memory Rules

RAM is state, not just another combinational expression. A read delayed by one clock might observe a newer write, even if its address is identical. Retiming therefore leaves the read transaction at its original clock and can register the value already read. Arithmetic and sidebands downstream can then be pipelined together. Write address, data, enable, and clock are left unchanged.

The RAM regression writes and reads the same address on the same edge, uses changing input data and biases, and checks the old-data read contract. A separate latency-aware scoreboard prevents an accidentally delayed memory access from passing as a harmless output delay. This digital collision rule is not a promise about physical asynchronous RAM collisions.

Preserve a Module for Technology Mapping

Annotate a combinational module with its estimated **input-to-output delay in nanoseconds**. The synthesis retimer treats the whole module as one indivisible box, using this delay instead of adding up its internal operation estimates:

```

1 class [[clang::annotate("CPPHDL_KEEP_BOX=2.1")]] MultiplyBox
2 : public cpphdl::Module {
3 public:
4     _PORT(uint8_t) a_in;
5     _PORT(uint8_t) b_in;
6     _PORT(uint16_t) product_out = _ASSIGN(product_comb_func());
7

```

```

8  uint16_t product_comb_func() {
9      return uint16_t(a_in()) * uint16_t(b_in());
10 }
11 };

```

This number is propagation delay, **not clock-cycle latency**. This box has no internal registers and still computes a combinational result. The declaration does not add a register or turn a multi-cycle operation into a single-cycle one. Use the normal retiming rule on its enclosing module or on the whole design.

- `keep_behaviour_retiming` may move boundaries in surrounding logic but does not move a register through the box or modify its internal operations.
- `fit_pipeline_retiming` may insert aligned register banks at the box inputs and in logic after its outputs. It cannot split the box. All box inputs are aligned together, including control inputs; outputs share the declared delay.
- A 2.1 ns box leaves only the remaining stage budget for surrounding logic, after clock-to-Q, setup and any reset mux overhead. A smaller declared delay allows more surrounding logic in the same stage. An oversized box causes an explicit `indivisible keep box exceeds target period` error in fit mode.

The current contract supports **pure combinational modules**. Registers, RAM, host effects, nested/overlapping keep boxes, and combinational paths which leave and re-enter the same box are rejected. Registered or multi-cycle macros need a separate sequential timing contract; do not describe those with this annotation. Ordinary frontend lowering and constant simplification still apply. This keeps the live operation-level implementation, not the original source text or port spelling; boundaries are emitted as packed input/output buses.

`operations.v` includes the preserved module implementation and its instance. The internal generic mapper leaves explicitly declared implementations intact. It includes their real bodies in `gates.v` and also writes `keep_boxes.v`. Consequently `gates.v` is self-contained and simulatable: surrounding logic is gate-level, while kept modules retain operations such as `*` for subsequent DSP or ASIC technology mapping. Compile `gates.v` alone; do not also compile `keep_boxes.v`, which would duplicate module definitions. No unresolved blackbox is substituted for simulation.

`timing.json` lists each live kept instance, its generated module name and declared delay. `--delay-scale` scales built-in cell estimates only; an explicitly declared box delay stays unchanged. These numbers are user-provided timing contracts, not timing measurements or guarantees of DSP inference.

External Combinational Functions

`CPPHDL_BLACKBOX=module:delay_ns` declares an external function implementation instead of retaining a lowered body. The retimer treats each call as an atomic combinational operation, aligns its arguments, and can insert registers before or after it in fit mode. It cannot

insert registers inside it. A delay larger than the available stage budget is rejected. Keep-behaviour mode does not move registers through these opaque operations.

The delay may be zero when intentionally excluding an unfinished math block from timing. This does not make the block physically instantaneous and does not describe a registered/multicycle implementation. External module bodies must be supplied separately; their unknown internal gates are not included in area reports. See external math blackboxes for the interface contract and simulation requirements.

`synth/tests/retiming/keep_box.cpp` compares C++, the retimed native graph and Verilator over 4,096 transactions with resets and tags. It tests both retiming modes and two declared box delays. Structural checks require the multiplication and internal addition to remain in one unregistered module, keep parent binding arithmetic outside it, and retain exactly one instance through generic mapping.

Reports and Tests

The output directory contains:

- `timing.json`: per-rule before/after delay, target status, moved boundaries, inserted register bits, added latency, feedback scheduling and initiation interval.
- `retimed_graph.cc`: the transformed shared graph, also usable by the native graph emitter.
- `operations.v`: operation-level Verilog after retiming.
- `gates.v` and `gates.json`: internally mapped generic gates and a cell report.
- Logs and `manifest.json`, including the timing report.

The register-bit insertion count describes the transformation, not the final mapped flip-flop count; synthesis can remove constant and redundant bits.

```
1 cmake --build build --target cphhdl synth_retiming_checks
2 ctest --test-dir build --output-on-failure -R '^synth_.*retiming'
```

`logic_retiming.cpp` checks arithmetic, tags, nonzero reset values, exposed intermediate registers, and module selection through annotations and CLI rules. `ram_retiming.cpp` checks RAM ordering and downstream arithmetic. Both modes run against native C++ and Verilated gate-level output. The regressions compare against cycle-level C++ reference models; there is no external SAT-tool dependency and these runtime checks are not formal proofs. `Checks.cpp` exercises delay scaling, impossible targets, invalid scopes, feedback, memory-write barriers, clock-domain rejection, initial-value preservation, and timing after bit-level dependency splitting.

`one_clock.cpp` checks two calls, native/gate equivalence, reset, and rejection of oversized or effectful functions. `feedback_frames.cpp` uses the same CRC and framing primitives as the

HFT application through `ReceiveMetadata.h`. Its native and gate tests cover 1,000 frames, validity bubbles, aligned size/CRC errors and frame metadata, changing inputs while busy, and reset during an in-flight transaction at a 312 MHz estimated target. It is a retimed frame-metadata stage, not the complete HFT packet parser or order generator.

The HFT pipeline example instead uses three `ClockedPipeline` regions and transfers words on physical clock edges, without the multicycle ready/commit adapter. Its four- and eight-stage RTL regressions pass continuous RX and intra-frame TX throughput checks. Its original 8.32 ns collector-to-decision boundary was outside the HLS regions. Manual RTL stages now separate framing, accumulation, checksum folding, validation and sequence filtering, with matching valid/error metadata. Full-design retiming estimates 3.15 ns against the 315 MHz target and keeps $H=1$. The gate regression checks packet results, stalls and reset after retiming. A region meeting timing alone does not establish timing for the logic that feeds it; the design-wide check still applies, and physical timing closure requires technology-specific tools.